ExamGuard — Technical Documentation & Viva Defense Handbook

Easy-to-Study Architecture, Codebase Walkthroughs, Cryptographic Specs & 50+ Viva Questions

ExamGuard — Technical Documentation & Viva Defense Handbook

An Easy-to-Study Architecture Guide, Concept-by-Concept Reference & Viva Question Bank

---

Handbook Overview & Defense Preparation Guide

This technical handbook is prepared specifically to help you explain, demonstrate, and defend **ExamGuard (Secure Examination Question Paper Management System)** before your project guide, department teachers, and external technical viva examiners.

Every technical concept in this document follows an **easy-to-study, structured format**:

1. **What is it?** — Explained in plain, simple English first.
2. **Why did we use it?** — The exact practical problem it solves.
3. **How does ExamGuard use it?** — Concrete implementation in this project.
4. **Where is it implemented?** — Exact file paths and function names from the actual codebase.
5. **How does it work?** — Step-by-step execution flow.
6. **Possible Viva Question & Model Answer** — Ready to speak aloud to examiners.
7. **Examiner Follow-Up Question** — Anticipating deeper technical cross-questioning.

Table of Contents

1. [Project Introduction & Problem Statement](#1-project-introduction--problem-statement)
2. [Project Objectives & Core Capabilities](#2-project-objectives--core-capabilities)
3. [System Architecture & Technology Stack](#3-system-architecture--technology-stack)
4. [Frontend, Backend & Database Architecture](#4-frontend-backend--database-architecture)
5. [Database Schema, Tables & Relationships](#5-database-schema-tables--relationships)
6. [Complete REST API Reference](#6-complete-rest-api-reference)
7. [Authentication: bcrypt, JWT & RBAC](#7-authentication-bcrypt-jwt--rbac)
8. [Symmetric Encryption: AES-256-GCM](#8-symmetric-encryption-aes-256-gcm)
9. [Asymmetric Key Wrapping: RSA-OAEP](#9-asymmetric-key-wrapping-rsa-oaep)
10. [Digital Signatures: RSA-PSS with SHA-256](#10-digital-signatures-rsa-pss-with-sha-256)
11. [Cryptographic Integrity: SHA-256 Digests](#11-cryptographic-integrity-sha-256-digests)
12. [The Canonical Verify-Before-Decrypt Pipeline](#12-the-canonical-verify-before-decrypt-pipeline)
13. [Secure Private PDF Storage & Path Security](#13-secure-private-pdf-storage--path-security)

---

14. [Automated Question Paper Generation (Module 18)](#14-automated-question-paper-generation-module-18)
15. [Post-Exam Answer Key Timing Windows (Module 10)](#15-post-exam-answer-key-timing-windows-module-10)
16. [In-App Notifications & Delivery Scheduling (Module 12)](#16-in-app-notifications--delivery-scheduling-module-12)
17. [Immutable Audit Logging & Admin Governance (Modules 13 & 14)](#17-immutable-audit-logging--admin-governance-modules-13--14)
18. [Secure Question Paper Deletion (Module 20)](#18-secure-question-paper-deletion-module-20)
19. [Testing, Verification & Quality Assurance](#19-testing-verification--quality-assurance)
20. [Cloud Deployment, Operations & Hardening](#20-cloud-deployment-operations--hardening)
21. [Limitations & Future Scope](#21-limitations--future-scope)
22. [12 Concrete Code-Defense Walkthroughs](#22-12-concrete-code-defense-walkthroughs)
23. [Comprehensive Technical Viva Question Bank (50+ Questions)](#23-comprehensive-technical-viva-question-bank)

1. Project Introduction & Problem Statement

What is it?

ExamGuard is a secure web application that handles confidential university examination question papers from the moment a teacher authors them to the moment they are printed in the examination center.

Why did we use it?

Traditional university paper handling relies on manual and insecure methods:

- Emailing PDF attachments across personal or institutional accounts.
- Storing unencrypted files on shared Google Drive or Dropbox links.
- Carrying papers on physical USB flash drives.
- Delivering printed copies days in advance to exam centers.

These methods lead to question paper leaks, undetectable modifications in transit, and post-exam answer key disputes.

How does ExamGuard use it?

ExamGuard eliminates plaintext handling completely. Plaintext papers exist only in volatile server memory during generation/download and are encrypted using military-grade envelope encryption (AES-256-GCM + RSA-OAEP). Access is time-locked so papers cannot be downloaded before their scheduled delivery date.

Possible Viva Question:

> ***Examiner:** *"Why is cloud storage like Google Drive or AWS S3 not enough for question papers?"**

> ***Model Answer:** "Standard cloud storage provides storage-level encryption where the cloud provider manages the keys. It does not provide application-level role authorization, time-locked scheduled delivery, verify-before-decrypt integrity pipelines, or digital signatures that prove non-repudiation by the authoring teacher."*

Follow-up Question:

> ***Examiner:** *"What prevents a database administrator who has direct access to Supabase from reading the papers?"**

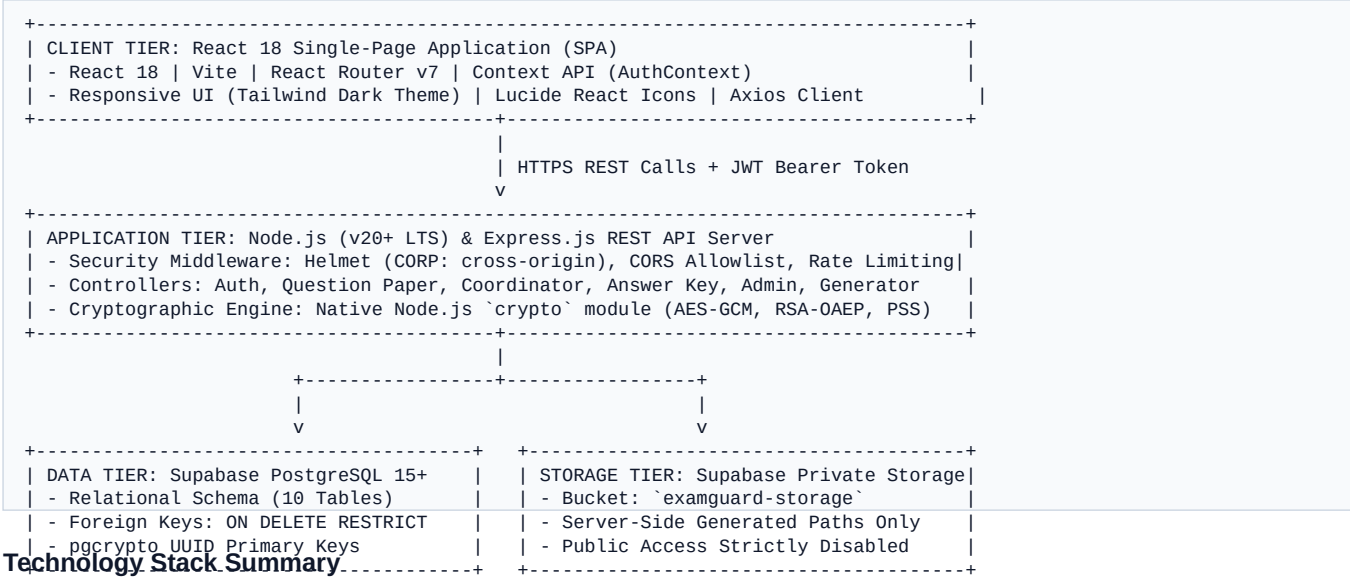
> ***Model Answer:** "The database stores only wrapped AES keys and metadata. The private RSA key required to unwrap those AES keys is stored separately on the backend application server host, not in the database. Even with full database access, an attacker cannot decrypt the files in storage."*

2. Project Objectives & Core Capabilities

- **Zero Plaintext Storage**: Plaintext PDFs are never saved to disk in either the database or object storage.

- **Envelope Encryption**: High-performance AES-256-GCM document encryption paired with RSA-OAEP asymmetric key wrapping.
- **Mathematical Non-Repudiation**: RSA-PSS digital signatures guarantee that any alteration to stored files is instantly detected.
- **Server-Controlled Delivery Dates**: Delivery dates are calculated on the server as `submission date + 1 UTC calendar day`, preventing client clock manipulation.
- **Automated University Paper Assembly**: Blueprint-driven generator that creates multi-set (Set A, Set B) papers matching university patterns with OR-alternatives and in-place question swapping.
- **Post-Exam Answer Key Timing**: Submission windows locked before the exam date and open for 0–4 days post-exam.
- **Immutable Audit Trail**: Append-only log with automated redaction of passwords, tokens, and cryptographic keys.

3. System Architecture & Technology Stack



Technology Stack Summary

Layer	Technology	Version	Purpose in ExamGuard
Frontend	React	18.3.1	Single-page user interface
Bundler	Vite	6.1.0	Fast client building & module bundling
Backend	Node.js / Express	v20+ / 4.21.2	REST API & cryptographic orchestration
Database	PostgreSQL (Supabase)	15+	Relational metadata, keys & audit logs
Storage	Supabase Storage	S3-compatible	Private bucket for encrypted `.enc` files
Symmetric Cipher	AES-256-GCM	NIST SP 800-38D	Document payload encryption
Key Encapsulation	RSA-OAEP	RFC 8017 (2048-bit)	Protecting symmetric DEK
Digital Signature	RSA-PSS	RFC 8017 (SHA-256)	Ciphertext integrity & non-repudiation

4. Frontend, Backend & Database Architecture

4.1 Frontend Architecture (`Frontend/src/`)

- **`App.jsx`**: Top-level router with `RootRedirect` to automatically navigate authenticated users to `/teacher`, `/exam-coordinator`, or `/admin`.
- **`context/AuthContext.jsx`**: Global authentication state provider managing `user`, `token`, `isAuthenticated`, and session persistence in `localStorage`.
- **`components/ProtectedRoute.jsx` & `RoleRoute.jsx`**: Client-side route guards preventing unauthorized navigation.

- **`components/QuestionPaperGeneratorWizard.jsx`**: Interactive 5-step wizard for syllabus blueprint setup, question bank curation, feasibility checking, multi-set previewing, and faculty approval.
- **`components/AlgorithmFlowDiagram.jsx`**: Interactive modal rendering the 11-step cryptographic pipeline and 5-step decryption sequence.

4.2 Backend Architecture (`Backend/src/`)

- **`server.js`**: Express server with Helmet security headers, CORS allowlist, and sanitized error handlers.
- **`config/envValidator.js`**: Validates mandatory environment variables at server boot.
- **`config/keyProvisioner.js`**: Decodes Base64 RSA keys on cloud instances.
- **`middleware/authMiddleware.js`**: Verifies JWT signatures and extracts user claims.
- **`middleware/roleMiddleware.js`**: Enforces RBAC permissions and verifies active account state in the database.
- **`middleware/uploadMiddleware.js`**: Multer memory storage with `%PDF-` magic-byte inspection.

5. Database Schema, Tables & Relationships

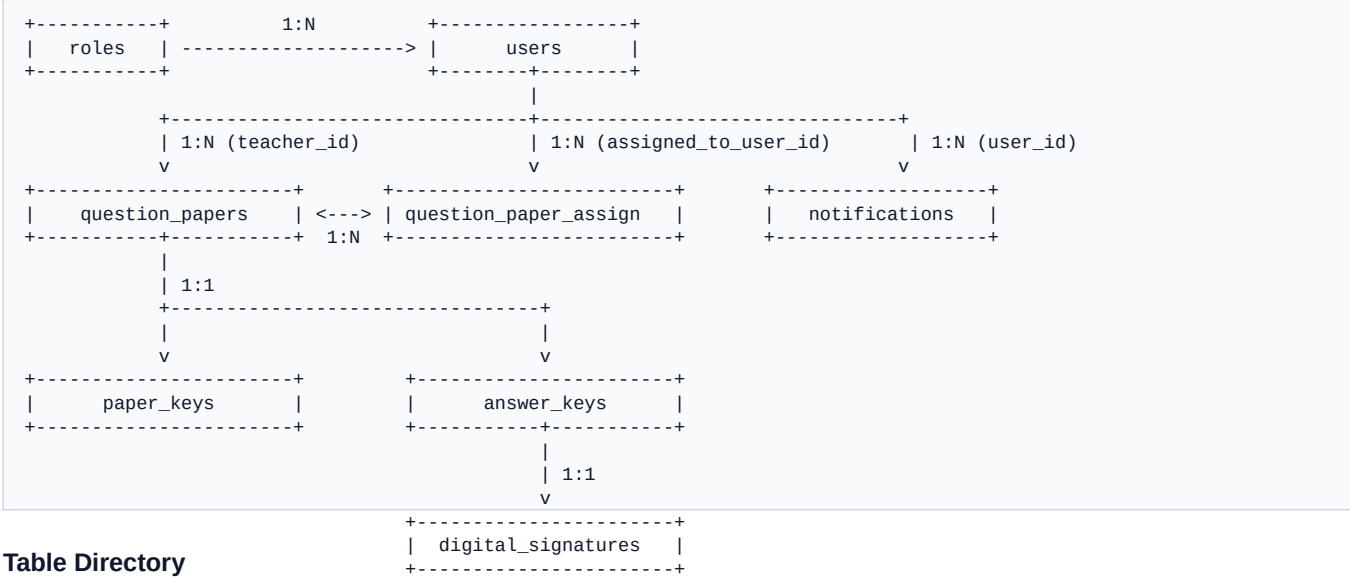


Table Directory

1. **`roles`**: System roles (`ADMIN`, `TEACHER`, `EXAM_COORDINATOR`).
2. **`users`**: User identities, bcrypt password hashes, and `is_active` state.
3. **`question_papers`**: Examination metadata, storage path, status, and scheduled delivery date.
4. **`question_paper_assignments`**: Maps papers to assigned reviewers and coordinators.
5. **`paper_keys`**: Stores RSA-wrapped AES key, 12-byte IV, and 16-byte GCM authentication tag ($1:1$ with `question_papers`).
6. **`answer_keys`**: Metadata for post-exam solution guides ($1:1$ with `question_papers`).
7. **`digital_signatures`**: Stores 64-char SHA-256 file hashes and RSA-PSS signatures.
8. **`notifications`**: In-app notifications with unread tracking.
9. **`activity_logs`**: Append-only security audit log with sanitized details.
10. **`question_bank` & `exam_blueprints`**: Categorized question pool and exam specifications (Module 18).

6. Complete REST API Reference

Endpoint	Method	Role	Description
`/api/health`	`GET`	Public	Liveness probe returning server health
`/api/health/ready`	`GET`	Public	Readiness probe checking database connectivity
`/api/auth/login`	`POST`	Public	Authenticates credentials; returns 24-hr signed JWT
`/api/auth/me`	`GET`	Authenticated	Returns profile of current logged-in user
`/api/question-papers`	`POST`	`TEACHER`	Uploads, encrypts, signs, and schedules PDF paper
`/api/question-papers`	`GET`	`TEACHER`	Lists papers authored by logged-in teacher
`/api/question-papers/assigned-to-me`	`GET`	`TEACHER`	Lists papers assigned to teacher for review
`/api/question-papers/:id/verify-signature`	`GET`	`TEACHER`	Inline RSA-PSS signature verification
`/api/question-papers/:id/download`	`GET`	`TEACHER`	Decrypts and streams paper (gated for reviewers)
`/api/question-papers/:id`	`DELETE`	`TEACHER`	Storage-first soft-deletion (retention guarded)
`/api/exam-coordinator/question-papers`	`GET`	`EXAM_COORDINATOR`	Lists papers assigned to coordinator
`/api/exam-coordinator/question-papers/:id/download`	`GET`	`EXAM_COORDINATOR`	Verified download (gated until delivery date)
`/api/teacher/question-papers/:id/answer-key`	`POST`	`TEACHER`	Uploads encrypted answer key in open window
`/api/notifications`	`GET`	`TEACHER`/`COORD`	Retrieves in-app notifications
`/api/audit-logs`	`GET`	`ADMIN`	Queries immutable audit logs with multi-filters
`/api/audit-logs/summary`	`GET`	`ADMIN`	Returns KPI metric counts (total, success, denied)
`/api/admin/users`	`POST`	`ADMIN`	Provisions new user with bcrypt password hash
`/api/admin/users/:id/status`	`PATCH`	`ADMIN`	Toggles account status (safeguard protected)
`/api/admin/users/:id/role`	`PATCH`	`ADMIN`	Changes user role (safeguard protected)
`/api/question-paper-generator/generate`	`POST`	`TEACHER`	Assembles Set A & Set B from blueprint

7. Authentication: bcrypt, JWT & RBAC

What is it?

The security framework that verifies who you are (**Authentication** via bcrypt and JWT) and what you are allowed to do (**Authorization** via Role-Based Access Control).

Why did we use it?

- Passwords must never be stored in plain text.
- Traditional server sessions require database reads on every request, increasing latency.
- Deactivated users must be blocked immediately without waiting for tokens to expire.

How does ExamGuard use it?

- Passwords are encrypted using **bcrypt** with a cost factor of **12 rounds**.

- The server generates stateless **JSON Web Tokens (JWT)** signed with an HMAC-SHA256 server secret (`JWT_SECRET`) valid for 24 hours.
- Every protected route passes through `roleMiddleware.js`, which queries `users.is_active` in real time. If an admin deactivates an account, access is revoked on the very next click.

Where is it implemented?

- Hashing: `Backend/src/services/authService.js` (`login`, `bcrypt.compare`).
- JWT Signing: `Backend/src/utils/jwt.js` (`generateToken`, `verifyToken`).
- RBAC Middleware: `Backend/src/middleware/roleMiddleware.js` (`authorizeRoles`).

Possible Viva Question:

> ***Examiner:** *"If JWT is stateless, how does ExamGuard immediately revoke access when an admin deactivates a user?"**

> ***Model Answer:** "While the JWT signature is verified statelessly by `authMiddleware.js`, our `roleMiddleware.js` performs an authoritative database lookup on `users.is_active`. If `is_active === false`, the request is immediately rejected with HTTP 403 Forbidden, revoking access without waiting for token expiration."*

---

8. Symmetric Encryption: AES-256-GCM

What is it?

A fast, military-grade symmetric cipher that encrypts large files using a 256-bit secret key. GCM stands for Galois/Counter Mode, which includes built-in tamper detection.

Why did we use it?

- High speed: Capable of encrypting multi-megabyte PDFs in milliseconds.
- Built-in Authentication: Generates a 16-byte authentication tag that detects any modification to the ciphertext or initialization vector (IV).

How does ExamGuard use it?

Every single question paper gets a fresh, randomly generated 32-byte AES key and a 12-byte IV. The PDF buffer is encrypted into `.enc` ciphertext. If any bit is flipped in storage, decryption fails immediately.

Where is it implemented?

- File: `Backend/src/services/encryptionService.js`
- Functions: `encryptQuestionPaper`, `decryptQuestionPaper`

How does it work?

```
[ Plaintext PDF Buffer ]
           %
           %¼
[ crypto.createCipheriv('aes-256-gcm', aesKey, iv) ]
           %
           % % %
```

Why did we use it?

RSA cannot encrypt large PDF files directly (RSA-2048 max payload is only 190 bytes with OAEP). Therefore, we use **Envelope Encryption**: AES encrypts the PDF, and RSA-OAEP encrypts the AES key.

How does ExamGuard use it?

The 32-byte AES key is wrapped with the institution's public key (`question-paper-key-wrapping-public.pem`) and stored in `paper_keys.encrypted_aes_key`. Only the backend server holding the private wrapping key can unwrap it.

Where is it implemented?

- File: `Backend/src/services/keyManagementService.js`
- Functions: `wrapDocumentKey`, `unwrapDocumentKey`

Possible Viva Question:

> ***Examiner:** *"Why use OAEP padding instead of PKCS#1 v1.5 padding?"**

> ***Model Answer:** "PKCS#1 v1.5 padding is vulnerable to Bleichenbacher's chosen-ciphertext padding attack. RSA-OAEP (Optimal Asymmetric Encryption Padding) incorporates random masks and SHA-256 hashing, providing provable IND-CCA2 security against chosen-ciphertext attacks."*

---

10. Digital Signatures: RSA-PSS with SHA-256

What is it?

A cryptographic seal placed on the encrypted file that proves who created it and guarantees it has not been modified.

Why did we use it?

Encryption scrambles data, but digital signatures provide **Non-Repudiation**: mathematical proof that the file came from the authorized source and was not replaced or corrupted.

How does ExamGuard use it?

The system hashes the encrypted ciphertext buffer using SHA-256 and signs the digest using the institution's private signing key (`question-paper-signing-private.pem`) with RSA-PSS padding.

Where is it implemented?

- File: `Backend/src/services/signatureService.js`
- Functions: `signQuestionPaper`, `verifyQuestionPaperSignature`

Possible Viva Question:

> ***Examiner:** *"What exact data is passed to the digital signature generator: plaintext or ciphertext?"**

> ***Model Answer:** "The exact encrypted ciphertext buffer is signed, never the plaintext. Signing the ciphertext enables our Verify-Before-Decrypt architecture: we can verify document authenticity and detect storage tampering before performing any key unwrapping or decryption."*

---

11. Cryptographic Integrity: SHA-256 Digests

What is it?

A one-way mathematical function that converts any file into an exact 64-character hexadecimal fingerprint.

Why did we use it?

The Avalanche Effect: If even a single comma or byte in the file is modified, the SHA-256 digest changes completely, exposing tampering.

How does ExamGuard use it?

Stored in `digital_signatures.file_hash`. Verified during inline signature checks and as Step 1 of the decryption pipeline.

---

12. The Canonical Verify-Before-Decrypt Pipeline

What is it?

The mandatory 5-step sequence enforced on the server before any question paper or answer key can be decrypted:

```
[ Ciphertext Buffer in Storage ]
                %
                %¼
   [ 1. SHA-256 Hash Check ] % % % % % % % %  Fail % % > [ ABORT: Tampered ]
                %  Pass
                %¼
   [ 2. RSA-PSS Signature Check ] % % %  Fail % % > [ ABORT: Bad Signature ]
                %  Pass
                %¼
   [ 3. RSA-OAEP Key Unwrapping ] % % %  Fail % % > [ ABORT: Corrupted Key ]
                %  Pass (32-byte DEK)
                %¼
   [ 4. AES-256-GCM Decryption ] % % % %  Fail % % > [ ABORT: Auth Tag Failure ]
                %  Pass
                %¼
   [ 5. Ephemeral Buffer Zeroization (`zeroizeBuffer(aesKey)`) ]
                %
                %¼
   [ Stream Authenticated Plaintext PDF ]
```

Why is it Fail-Closed?

If any step fails, execution halts immediately with HTTP 400. No key is unwrapped, no decryption occurs, and zero bytes of plaintext are emitted.

Where is it implemented?

- File: `Backend/src/services/verificationService.js`
- Function: `verifyBeforeDecryptQuestionPaper`

---

13. Secure Private PDF Storage & Path Security

- **Private Bucket**: `examguard-storage` in Supabase Storage. Direct public URL access is strictly disabled.
- **Server-Generated Paths**: `question-papers/{teacher_id}/{paper_id}/encrypted/{filename}.enc`.
- **Traversal Defense**: Client-supplied paths and sequence tokens (`..`, leading slashes) are strictly rejected.

---

14. Automated Question Paper Generation (Module 18)

- **Curriculum Blueprint**: Specifies subject code, duration (2.5 hrs), total marks (70), instructions, and university OR-group sections.
- **Feasibility Engine**: Validates that the question pool has sufficient depth without exceeding the 40% maximum set overlap limit.
- **PDFKit University Renderer**: Standardized university format with header, Seat Number box, and right-aligned mark brackets (`Backend/src/services/pdfRendererService.js`).
- **In-Place Swapping**: Allows faculty to swap any question in Set A or Set B with an alternative matching the exact unit, marks, and difficulty.

---

15. Post-Exam Answer Key Timing Windows (Module 10)

- Server calculates: $\text{diffDays} = \text{CurrentDate} - \text{ExamDate}$.
- $\text{diffDays} < 0$: Status `BEFORE_EXAM` $\rightarrow$ upload blocked (HTTP 400).
- $0 \le \text{diffDays} \le 2$: Status `WINDOW_OPEN_PREFERRED` $\rightarrow$ upload open.
- $2 < \text{diffDays} \le 4$: Status `WINDOW_OPEN_EXTENDED` $\rightarrow$ upload open.
- $\text{diffDays} > 4$: Status `WINDOW_EXPIRED` $\rightarrow$ upload blocked (HTTP 400).
- Packaged in a **self-contained 284-byte container** `[256-B wrapped DEK + 12-B IV + 16-B AuthTag + Ciphertext]` signed with RSA-PSS.

---

16. In-App Notifications & Delivery Scheduling (Module 12)

- Calculates: $\text{Scheduled Delivery Date} = \text{Submission Date} + 1\text{ Day (UTC)}$.
- Download requests from coordinators before this date return HTTP 403 Forbidden.
- Dispatches in-app notification records to `notifications` table with idempotency checks to avoid duplicate alerts upon retry.

17. Immutable Audit Logging & Admin Governance (Modules 13 & 14)

- **Audit Logging**: `Backend/src/services/auditLogService.js` records high-value events in `activity_logs`.
- **Automatic Redaction**: Recursively strips passwords, tokens, AES keys, and storage paths.
- **Admin Safeguards**: `Backend/src/services/adminService.js` prevents self-lockout, self-demotion, and deactivating the last active administrator account.

18. Secure Question Paper Deletion (Module 20)

- **Retention Locks**: Prevents deletion if paper status is `DELIVERED`, if assignments are `DELIVERED`, or if an answer key exists (HTTP 409 Conflict).
- **Storage-First Erasure**: Deletes the encrypted ciphertext from private Supabase Storage *first*. If storage fails, database updates are aborted (Fail-Closed, HTTP 502).
- **Soft-Delete**: Updates `is_deleted = true`, `status = 'DELETED'`, `deleted_at = NOW()`.

19. Testing, Verification & Quality Assurance

- **18 Test Suites**: Located in `Backend/scripts/` covering authentication, RBAC, encryption, signatures, key management, coordinator access, answer keys, notifications, audit logs, admin governance, and deletion.
- **Unit Verification Evidence**:
- `testServiceLogic.js`: **8 passed, 0 failed**.
- `verifyEncryption.js` unit suite: **13 passed, 0 failed**.
- **Frontend Build Verification**: Vite production build succeeded cleanly in **11.69s** with zero syntax or compilation errors.

20. Cloud Deployment, Operations & Hardening

- **Frontend**: Deployed on Vercel (`https://examguardapp.vercel.app`).
- **Backend API**: Deployed on Render (`https://examguard-999h.onrender.com/api`).
- **Database**: Supabase PostgreSQL with `pgcrypto`.
- **Storage**: Supabase Private Bucket `examguard-storage`.
- **Security Headers**: Helmet with `Cross-Origin-Resource-Policy: cross-origin` for PDF blob streaming.
- **Rate Limiting**: `express-rate-limit` on `/api/auth/login` (10 attempts per 15 min per IP).

21. Limitations & Future Scope

- **Current Limitations**: Single private bucket segmented by folder; RSA keys stored via server host environment variables rather than dedicated Hardware Security Modules (HSM); in-app notifications only.
- **Future Scope**: Integration with FIPS 140-2 Level 3 Hardware Security Modules; biometric multi-factor authentication (MFA); dynamic examination watermarking.

22. 12 Concrete Code-Defense Walkthroughs

Walkthrough 1: User Login & JWT Token Issuance

1. **Entry**: Client `POST /api/auth/login` with `{ email, password }`.
2. **Controller**: `authController.login` invokes `authService.login(email, password)`.
3. **Database**: Queries `users` joined with `roles` where `email = :email`.
4. **Validation**: Checks `users.is_active === true`. Compares password with `bcrypt.compare(password, user.password_hash)`.
5. **Token**: `jwt.sign({ userId, email, role }, config.jwt.secret, { expiresIn: '24h' })`.
6. **Audit**: Inserts `LOGIN` event in `activity_logs`. Returns HTTP 200 with JWT.

Walkthrough 2: Protected API Request & RBAC Validation

1. **Header**: Reads `Authorization: Bearer <token>`.
2. **`authMiddleware`**: Verifies signature with `jwt.verify`. Attaches `req.user`.
3. **`roleMiddleware`**: Verifies `allowedRoles.includes(req.user.role)`. Queries database to ensure `users.is_active === true`. Passes to controller.

Walkthrough 3: Manual Paper Upload & Encryption

1. **Multer**: Buffers file; checks size $\le 10\text{ MB}$ and magic bytes `%PDF-`.
2. **AES Encryption**: `crypto.createCipheriv('aes-256-gcm', aesKey, iv)`. Produces ciphertext and 16-byte auth tag.
3. **RSA-OAEP Wrap**: `crypto.publicEncrypt(wrappingPubKey, aesKey)`.
4. **RSA-PSS Sign**: `crypto.createSign('sha256')`. Signs ciphertext buffer.
5. **Storage**: Uploads ciphertext to `examguard-storage`. Saves records in `question_papers`, `paper_keys`, and `digital_signatures`. Sets delivery date = tomorrow.

Walkthrough 4: Question Paper Generation Pipeline (Module 18)

1. **Snapshot**: Fetches approved structure from `generated_exam_sets`.
2. **PDFKit**: `pdfRendererService.renderQuestionPaperPDF` renders university header, Seat No. box, instructions, and OR-groups into an in-memory buffer.
3. **Encryption**: Piped directly into `encryptionService.encryptQuestionPaper`. Uploaded to storage. Plaintext is discarded.

Walkthrough 5: Coordinator Scoped Access & Delivery Gating

1. **Scoping**: Queries `question_paper_assignments` where `assigned_to_user_id = coordinatorId`.
2. **Gating**: Compares `todayStr < paper.scheduled_delivery_date`. If true, returns HTTP 403 Forbidden with delivery date notice.

Walkthrough 6: Inline Digital Signature Verification

1. **Retrieval**: Downloads ciphertext from storage; retrieves stored signature from `digital_signatures`.
2. **Verification**: Computes SHA-256 digest. Verifies with `crypto.createVerify('sha256')` using public signing key. Returns `{ verified: true }`.

Walkthrough 7: Verify-Before-Decrypt Download Pipeline

1. **Step 1**: Recomputes SHA-256 digest and compares with `digital_signatures.file_hash`.
2. **Step 2**: Verifies RSA-PSS digital signature over ciphertext.
3. **Step 3**: Unwraps AES key using private wrapping key via RSA-OAEP.

4. **Step 4**: Decrypts ciphertext with AES-256-GCM; authenticates 16-byte tag.
5. **Step 5**: `zeroizeBuffer(aesKey)`. Streams authenticated PDF buffer.

Walkthrough 8: Answer Key Submission Window Enforcement

1. **Ownership**: Verifies submitter is authoring teacher.
2. **Window**: Checks $\text{diffDays} = \text{CurrentDate} - \text{ExamDate}$. Blocks if before exam (<0) or expired (>4).
3. **Packaging**: Assembles 284-byte container `[256-B wrapped key + 12-B IV + 16-B tag + ciphertext]`. Signs with RSA-PSS.

Walkthrough 9: Admin User Provisioning & Safeguards

1. **Creation**: Admin calls `POST /api/admin/users`. Hashes password with bcrypt (12 rounds).
2. **Safeguard**: On status change, checks if target is requesting admin (self-lockout) or last active admin. Blocks with HTTP 400 if violated.

Walkthrough 10: Fail-Closed Question Paper Deletion

1. **Guards**: Checks status is not `DELIVERED` and no answer key exists.
2. **Storage Erasure**: Deletes file from Supabase Storage first. If storage fails, database update is aborted.
3. **Soft-Delete**: Sets `is_deleted = true`, `status = 'DELETED'`. Writes audit record.

---

23. Comprehensive Technical Viva Question Bank (50+ Questions)

Q1: What is Envelope Encryption and why is it used in ExamGuard?

Model Answer: "Envelope encryption is a hybrid cryptographic practice where a fast symmetric key (AES-256) encrypts the document payload, and an asymmetric key (RSA-2048) encrypts the symmetric key. We use it because asymmetric encryption cannot directly encrypt multi-megabyte PDF files, while symmetric encryption provides high speed and authenticated integrity."

Q2: Why did you choose AES-256-GCM over AES-CBC?

Model Answer: "AES-GCM is an Authenticated Encryption with Associated Data (AEAD) cipher. Unlike CBC mode, GCM does not require padding, eliminating padding oracle attacks like POODLE. Furthermore, GCM automatically produces a 16-byte authentication tag that detects any tampering with the ciphertext or IV."

Q3: Why is the RSA key pair for key wrapping separate from the digital signing key pair?

Model Answer: "This enforces the cryptographic Principle of Key Separation. Using the same key pair for both encryption and signing exposes the system to cross-protocol chosen-ciphertext attacks, where an attacker tricks an authority into signing a digest that decrypts sensitive data. Keeping them strictly segregated in `Backend/keys/` prevents this."

Q4: Walk through the Verify-Before-Decrypt pipeline. Why is it important?

Model Answer: "Verify-Before-Decrypt verifies the SHA-256 digest and RSA-PSS digital signature over the ciphertext *before* attempting any key unwrapping or decryption. This is a fail-closed architecture: if an attacker modifies the file in storage, we abort immediately with HTTP 400 without unwrapping keys or touching decryption routines."

Q5: What is ephemeral buffer zeroization and why does ExamGuard implement it?

Model Answer: "When a question paper is decrypted, the 32-byte AES key exists in server RAM. We implement `zeroizeBuffer(buf)` using `buf.fill(0)` inside `finally` blocks in `verificationService.js`. This actively overwrites the memory buffer with zeros, preventing key extraction via heap dumps or memory scraping."

Q6: How does ExamGuard prevent horizontal privilege escalation (IDOR)?

Model Answer: "We do not trust client parameters. In `questionPaperService.js` and `authorizationService.js`, every query explicitly checks whether the requesting user's ID matches `teacher_id` or appears in `question_paper_assignments`. If neither, the server returns HTTP 403 Forbidden and logs an `UNAUTHORIZED_ACCESS` event."

Q7: How does scheduled delivery date gating work?

Model Answer: "The server deterministically sets `scheduled_delivery_date = submission_date + 1 day (UTC)`. When an assigned coordinator requests a download, `examCoordinatorService.js` compares the current UTC date with the delivery date. If the date has not arrived, download is blocked with HTTP 403. Client clocks are completely ignored."

Q8: What are the answer key timing windows?

Model Answer: "To prevent leaks before exams, `answerKeyService.js` calculates calendar days since the exam date: before exam is strictly locked; Days 0–2 is the preferred window; Days 3–4 is the extended window; after Day 4, the window expires."

Q9: Why does question paper deletion follow a storage-first approach?

Model Answer: "To adhere to the fail-closed principle. If we deleted the database record first and storage deletion failed, an unmanaged, orphaned encrypted file would remain in storage. By deleting from storage first, any storage failure aborts the database update, maintaining system consistency."

Q10: What safeguards protect administrator accounts?

Model Answer: "`adminService.js` enforces three rules: an admin cannot deactivate themselves, an admin cannot demote themselves, and the last active admin account in the database can never be deactivated or demoted, preventing the system from becoming headless."

End of ExamGuard Technical Documentation & Viva Defense Handbook.